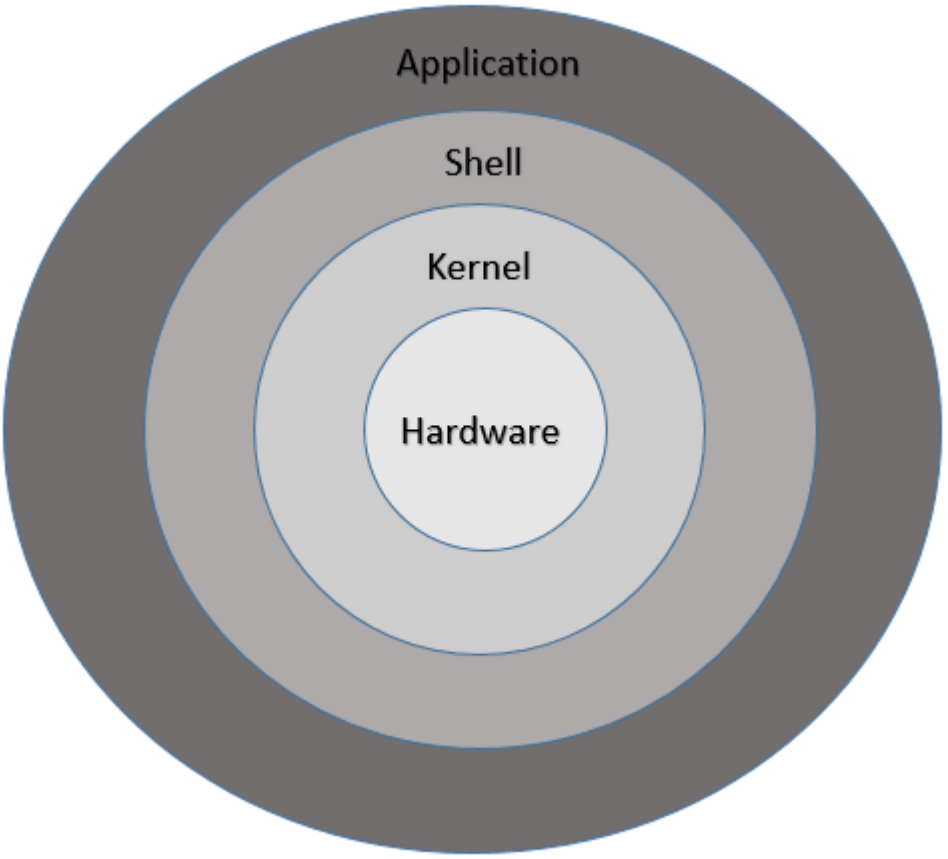
Module 0: Linux Essentials for Automation

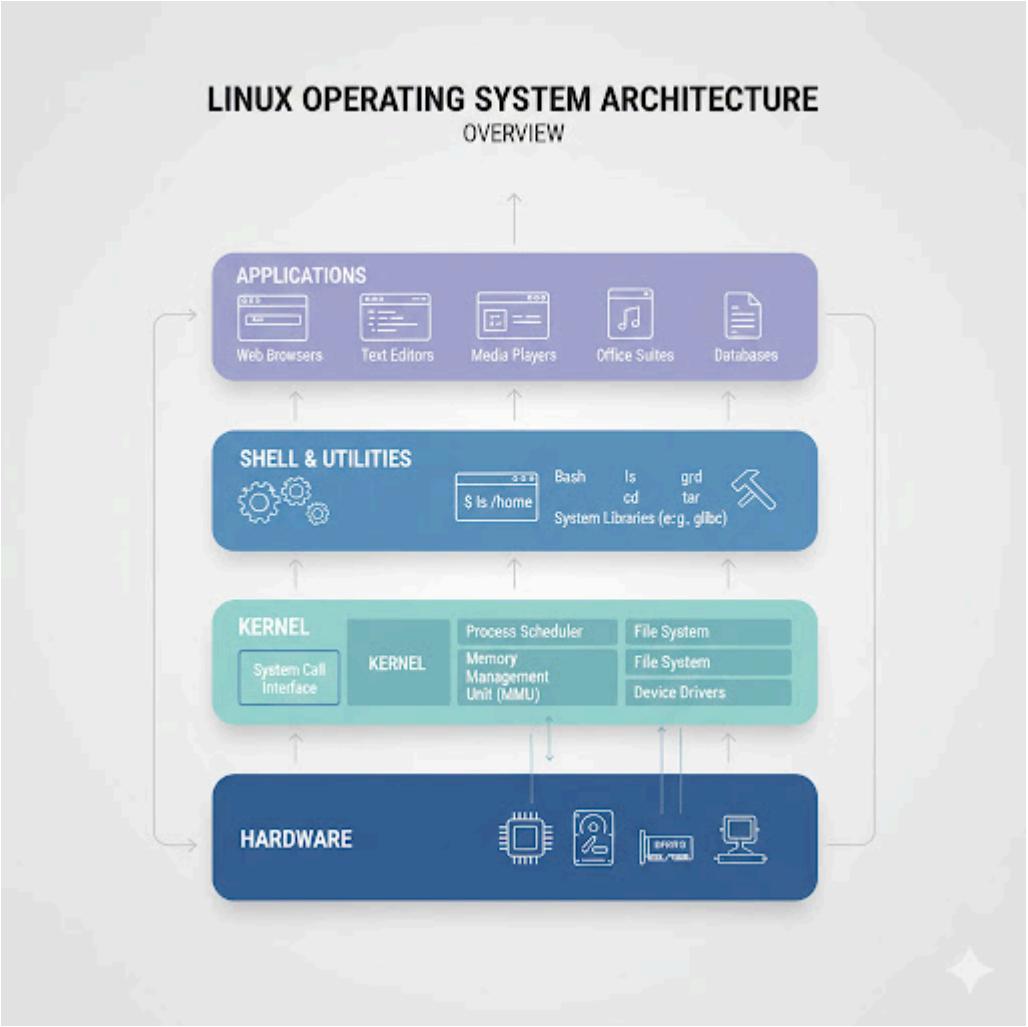
Linux Architecture & Shell Environment

What is Linux

- Multi-user, multitasking operating system
- Core of all modern cloud & DevOps systems
- Three key components:

    - **Kernel** → Manages hardware (CPU, memory, disk)
    - **Shell** → Interface between user & kernel
    - **User Space** → Applications, libraries, configurations





1. **Hardware:** Physical devices such as RAM, HDD, CPU, etc.
2. **Kernel: Core of the operating system that directly interacts with the hardware.** It handles processes, memory management, device drivers, and system calls.
3. **Shell & Utilities:** The interface between the user and the kernel. The shell (like Bash) allows users to execute commands. Utilities are small programs that perform specific tasks (e.g., `ls`, `cp`, `mv`). System libraries (like glibc) provide standard functions for applications.
4. **Applications:** User programs that run on top of the shell and utilities layer, using system resources provided by the kernel (e.g., web browsers, text editors, media players).

Shell Environment

- Command interpreter for executing tasks
- Converts user input → Kernel commands
- Used in automation scripts (Bash, Python, Ansible, Jenkins, etc.)

Common Shells Environment:
bash, sh, zsh, ksh

Task	Command
Check current shell	`echo $SHELL`
List available shells	`cat /etc/shells`
Check kernel version	`uname -r`
Full system info	`uname -a`
CPU & memory details	`lscpu`, `free -h`

Basic Linux commands

Here are some basic Linux commands that you may know must.

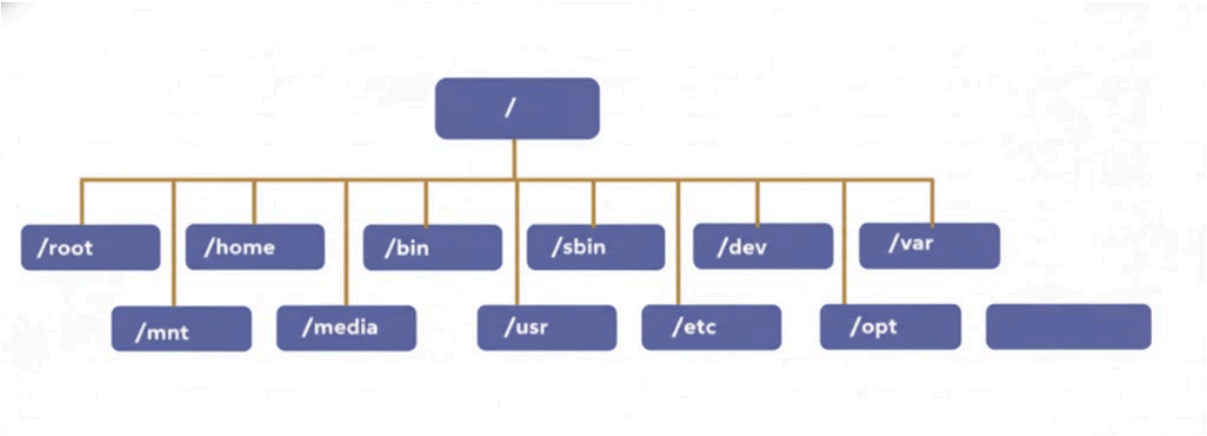
Basic Commands	
clear	Clear the screen
pwd	Show present working directory
ls	List the files or folders in the current directory
whoami	Shows currently logged in user
cd	Change directory
history	Displays a list of previously used commands
echo	Display text on the terminal
cat	Displays the content of a file
sudo	Superuser privileges
mkdir	create a directory
mkdir <dir1> <dir2>	Create multiple directories
mkdir <directory{1..5}	Creates multiple directories with same name up to 5 directories
touch <file.txt>	Create a single file
touch <file1.txt> <file2.txt>	Create multiple files
touch <files{1..5}.txt	Create multiple files with the same name up to 5 files
cp <source file or dir> <destination>	Copy and paste file or directory
rm <file.txt> or -r <folder>	Remove a file or folder

File System, Permissions, Users & Groups

File System Hierarchy:

Understanding the Linux File System

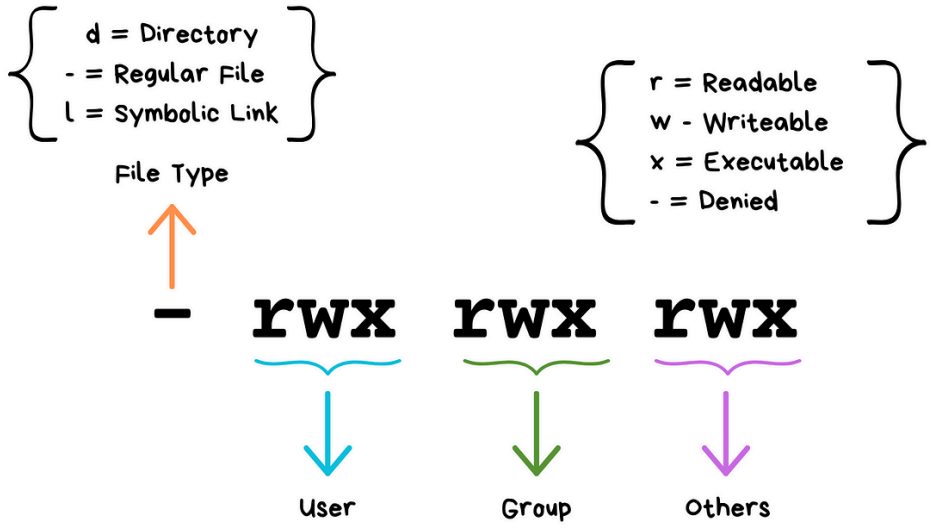
- Linux follows a **hierarchical file system structure**.
- Everything begins from the **root directory** **/**.
- Each directory has a specific purpose and stores particular types of files.



Key Directories in Linux

Directory	Purpose / Description
`/`	Root directory — the starting point of the entire file system
`/bin`	Essential user binaries (e.g., `ls`, `cat`, `cp`)
`/sbin`	System binaries used by administrators
`/usr/bin`	Additional user commands and applications
`/etc`	System-wide configuration files
`/home`	User home directories (personal data, configs)
`/var`	Variable data — logs, mail, databases
`/tmp`	Temporary files (auto-cleared after reboot)
`/opt`	Optional or third-party software
`/dev`	Device files representing hardware components
`/proc`	Virtual directory showing process & kernel info

File Permissions and Ownership:



Understanding File Permissions

- Every file or directory in Linux has **three levels of access control**:
  - **Owner (User)** → The file creator or assigned owner
  - **Group** → Users belonging to the same group as the file
  - **Others** → All other users on the system
- Each access type is represented by:
  - **r → Read**
  - **w → Write**
  - **x → Execute**
- The first character in `ls -l` output shows **file type**:
  - `d` → Directory
  - `-` → Regular file

Example:

drwxr-xr-- 2 ubuntu devops 4096 Oct 22 scripts

Meaning of Permissions

Permission Type	Symbol	Description
Read	`r`	View file content or list directory contents
Write	`w`	Modify file or create/delete items in a directory
Execute	`x`	Run a file or access directory contents

Without `x` permission, you can't enter (`cd`) into a directory even if you have `r` permission.

Viewing Permissions

Command:

ls -l

Example Output:

-rwxr-xr-- 1 ubuntu ubuntu 1024 Oct 22 test-file.txt

Breakdown:

- `-` → Regular file
- `rwx` → Owner has read, write, execute
- `r-x` → Group has read, execute
- `r--` → Others have only read

Changing Permissions with chmod

- **chmod** (Change Mode) modifies access permissions.
- Permission values are based on binary sum:
    - `r = 4`, `w = 2`, `x = 1`

Code	Meaning
7	Full (rwx)
6	Read + Write
5	Read + Execute
4	Read only
0	No permission

Chmod Practical Examples

Command	Meaning
`chmod 777 file.txt`	Everyone can read, write, execute
`chmod 600 file.txt`	Owner: read/write; Group & Others: none
`chmod 750 file.txt`	Owner: rwx; Group: r-x; Others: none
`chmod 700 file.txt`	Owner: rwx; Group/Others: no access
`chmod 444 file.txt`	All users: read-only (no modify or execute)
`chmod 755 file.txt`	Owner: rwx; Group/Others: r-x (most common for scripts)

Use Case:
`chmod 755 deploy.sh` → Allows everyone to execute the deployment script, but only the owner can modify it.

Understanding chmod 755 in Detail

Command:

chmod 755 filename

Breakdown:

Digit	Applies To	Permission
7	Owner	Read + Write + Execute
5	Group	Read + Execute
5	Others	Read + Execute

Use Case: Used for scripts and executables that need to be run by any user but not modified.

Changing File Ownership (chown)

- **chown** (Change Owner) changes file or directory ownership.
- Syntax:

sudo chown new_owner filename

Example:

sudo chown niloy test-file.txt

Explanation:

- Changes the owner of `test-file.txt` from `ubuntu` to `niloy`
- If the user `niloy` doesn't exist, you'll get an "invalid user" error
  → Create the user first using `sudo adduser niloy`

After success:

ls -l

You'll see the owner updated to `niloy`.

Changing Group Ownership (chgrp)

- **chgrp** (Change Group) changes group ownership of a file or directory.
- Syntax:

sudo chgrp new_group filename

Example:

sudo chgrp niloy test-file.txt

💡 *Explanation:*

- Changes the group ownership of the file to `niloy`
- When a new user is created, a default group with the same name is automatically created

Verification:

id niloy

→ Confirms `niloy` group exists.

File & Group Ownership Use Cases

Action	Command	Use Case
Change user ownership	`chown`	Transfer file control to another user
Change group ownership	`chgrp`	Assign group-level access for collaboration
Change both	`chown user:group file`	Change both owner and group together

Example:

sudo chown niloy:devteam project.txt

→ Owner becomes `niloy`, group becomes `devteam`.

Key Takeaways

✔ **Permissions** define what users can read, write, or execute
✔ **Ownership** determines who controls the file
✔ **chmod**, **chown**, and **chgrp** are core Linux administration tools

Linux Command-line Data Processing

---

grep – Global Regular Expression Print

- Search and filter lines that match specific patterns.

Example	Description
`grep "ERROR" app.log`	Show lines containing "ERROR"
`grep -c "ERROR" app.log`	Count number of "ERROR" entries
`grep -i "info" app.log`	Case-insensitive search
`grep "ERROR" *.log`	Search pattern across multiple files

awk – Text & Data Extraction

- Process data field-by-field (column-wise).

sed – Stream Editor

- Perform find-and-replace or text modification directly inside files.

find – File Search & Cleanup Utility

- Locate files based on name, type, size, or modified time.

cut – Extract Specific Fields

- Extract columns from text or CSV data.

Combining Commands

- Combine multiple tools for powerful data workflows.

Example 1 – Grep + Cut

grep "dev" users.csv | cut -d',' -f2

➡️ Extracts only names of Dev/DevOps users.

Example 2 – Grep + Awk

grep "devops" users.csv | awk -F',' '{print $2}'

➡️ Displays DevOps team names cleanly.

Step-by-step Guide:

Lab folder (already created in your example): `~/linux-lab` with `app.log` and `users.csv`.
Work from that folder:

cd ~/linux-lab

pwd # check you are in the lab folder

ls -l

🔹 Part A — `grep` : search & filter logs

Lab A1 — Find `ERROR` lines

Command:

grep "ERROR" app.log

Line-by-line explain

- `grep` → the command name (Global Regular Expression Print) — it searches text for pattern.
- `"ERROR"` → the pattern (literal string) we search for.
- `app.log` → file to search in.

What happens: `grep` reads each line of `app.log` and prints lines that contain `ERROR`.

Expected output (from your app.log):

2025-10-21 10:01:45 ERROR Database connection failed

2025-10-21 10:05:10 ERROR Disk space full

Why use: quick way to find error occurrences in logs.

Lab A2 — Count occurrences

Command:

grep -c "ERROR" app.log

Lab A3 — Case-insensitive search

Command:

grep -i "info" app.log

Explain

- `-i` → ignore case (INFO, Info, info all match).
  Output:

2025-10-21 10:00:01 INFO Server started

2025-10-21 10:04:12 INFO New user registered: nahid

Lab A4 — Search across multiple files

Suppose you have many `*.log` files:

grep "ERROR" *.log

Explain

- `*.log` → shell expands to all files ending with `.log` in current dir.
- Output will show filename:line format when multiple files present.

🔹 Part B — `awk` : field/column processing

`awk` splits each line into fields and allows processing.

Lab B1 — Print second column from CSV (names)

Command:

awk -F',' '{print $2}' users.csv

Explain

- `awk` → program to process lines/fields.
- `-F','` → set field separator to comma `,` (CSV).
- `{print $2}` → print the second field of each line (`$1` = first field).

Why: extract the `name` column quickly.

Lab B2 — Filter rows where role == "devops" and print name & role

Command:

awk -F',' '$4 == "devops" {print $2, $4}' users.csv

Explain

- `$4 == "devops"` → condition: fourth field equals string `devops`.
- `{print $2, $4}` → action when condition true: print second and fourth fields, separated by a space.

Why: role-based extraction for access control or reports.

Lab B3 — Skip header (NR>1)

Often CSV has header; skip it:

awk -F',' 'NR>1 && $4=="dev" {print $2}' users.csv

Explain

- `NR` → current record (line) number.
- `NR>1` → skip first line (header).
- `&&` → logical AND.

Why: avoid treating header as data.

🔹 Part C — `sed` : stream editor for inline edits

Lab C1 — Preview replace (no file change)

sed 's/ERROR/CRITICAL/g' app.log

Explain

- `sed` → stream editor.
- `'s/ERROR/CRITICAL/g'` → substitution: `s` command, replace `ERROR` with `CRITICAL`, `g` = global (all occurrences on line).
- Output prints modified content to stdout (screen), original file unchanged.

Why preview before changing file.

Lab C2 — Edit file in place (overwrite)

CAUTION: modifies file

sed -i 's/ERROR/CRITICAL/g' app.log

Explain

- `-i` → edit file in place (modify `app.log`).
  Afterwards, `cat app.log` will show `CRITICAL` instead of `ERROR`.

Why use carefully: use a backup option `-i.bak` to keep original:

sed -i.bak 's/ERROR/CRITICAL/g' app.log

This creates `app.log.bak`.

Lab C3 — Delete lines matching a pattern

sed -i '/WARN/d' app.log

Explain

- `'/WARN/d'` → for lines matching `WARN`, delete (`d`).
- `-i` modifies file.

Why: quick cleaning before archiving logs.

Safety tip

Always test `sed` without `-i` first. Use backups.

🔹 Part D — `find` : locate files & act

Lab D1 — Find all `.log` files in current directory tree

find . -type f -name "*.log"

Explain

- `find` → search utility.
- `.` → start path (current dir).
- `-type f` → only files.
- `-name "*.log"` → filename pattern (wildcard).

Output: list of matching paths, e.g. `./test.log`.

Lab D2 — Find and delete files older than 7 days

CAUTION: destructive — run without `-delete` to test first
Test:

find /var/log -type f -mtime +7 -print

- `-mtime +7` → files modified more than 7 days ago.
  If output correct, then delete:

find /var/log -type f -mtime +7 -delete

Why: automated log rotation/cleanup (cron job).

Lab D3 — Find large files (>10M)

sudo find / -type f -size +10M -print

Explain

- `-size +10M` → files bigger than 10 megabytes.
- `sudo` may be required to access protected dirs.

Why: free up disk space in production.

Lab D4 — Find and archive old logs (combine with `tar`)

find . -type f -name "*.log" -mtime +7 -print0 | tar --null -T - -rvf old_logs.tar

Explain

- `-print0` and `--null` + `-T -` makes it safe for filenames with spaces.
- `-rvf` → append to archive verbosely.

Why: safe archiving in script.

🔹 Part E — `cut` : extract fixed fields

Lab E1 — Extract 3rd column (email) from CSV

cut -d',' -f3 users.csv

Explain

- `cut` → extract text columns.
- `-d','` → delimiter comma.
- `-f3` → third field.

Output:

email

nahid@example.com

rahim@example.com

karim@example.com

anika@example.com

Why: export emails for notifications.

Lab E2 — Combine `grep` + `cut` (filter then extract)

Correct usage:

grep 'dev' users.csv | cut -d',' -f2

Explain

- `grep 'dev' users.csv` → filter lines containing `dev`.
- `|` → pipe output of grep into cut.
- `cut -d',' -f2` → extract second column (name).

Be careful: Do not run `grep 'dev' | cut -d',' -f2 users.csv` — that pipes nothing into grep and incorrectly supplies file to `cut`. Order matters: `grep <file> | cut ...`

Your terminal confusion fixed: earlier you ran `grep 'dev' | cut -d',' -f2 user.csv` — wrong order. Correct is `grep 'dev' user.csv | cut -d',' -f2`.

Why combine: efficient one-liner for pipelines.

Part F — `awk` + `grep` + `cut` examples (practical combos)

Example 1 — Extract emails for `dev` and save to file

grep ',dev,' users.csv | cut -d',' -f3 > dev_emails.txt

Explain

- `',dev,'` pattern ensures role column equal `dev` (safer than `dev` alone).
- Output redirected `>` into `dev_emails.txt`.

Example 2 — Extract date/time from CRITICAL lines (from app.log)

grep "CRITICAL" app.log | awk '{print $1, $2}' > critical_times.txt

Explain

- `grep "CRITICAL" app.log` -> select lines.
- `awk '{print $1, $2}'` -> print first two fields (date & time).
- Save to `critical_times.txt`.

Processes, Services & Logs

Concept Explanation

- A **Process** is a running instance of a program in Linux.
- A **Service** is a background process that usually starts automatically at boot (e.g., `sshd`, `crond`).
- Each process is identified by a **PID (Process ID)**.
- Services are managed by **systemd** using `systemctl`.

Process Monitoring - Example

```
# View running process
ps aux | grep nginx

# Kill a process
sudo kill -9 <PID>

# Check service status
sudo systemctl status ssh
sudo systemctl restart nginx
```

Networking Essentials &Troubleshooting

Concept Explanation

- Networking tools help verify **connectivity**, **service health**, and **firewall or security group issues**.
- Commonly used in **DevOps troubleshooting** and **CI/CD health checks**.
- `curl`, `netstat`, and `ping` are frequently used during deployment validation.

Environment Variables

Concept Explanation

- **Environment variables** define the execution context of processes.
- They are temporary key-value pairs used by applications and automation scripts.
- Example: `$PATH`, `$HOME`, `$USER`, `$SHELL`.

Usage Examples

export DB_HOST=prod-db.example.com

echo $DB_HOST

Use Case:

- CI/CD pipelines (Jenkins, GitHub Actions) store credentials and API tokens as environment variables.
- Example: `$DB_PASS`, `$API_KEY`, `$AWS_REGION`.

Module 1 | Bash Scripting Fundamentals

Shell Scripting Basics — Shebang & Variables

Introduction to Shell Scripting

What is Shell?

- A program that executes user commands and communicates with the OS
- Converts human-readable commands → machine-level instructions

Example:

ls → Shell → Kernel → Output: file list

Common Shells:

- `bash` → Most common (Linux default)
- `zsh` → Used in macOS
- `PowerShell` → Windows environment

Shell Prompt Overview

kali@linux:~$

- `kali` → Username
- `linux` → Hostname
- `~` → Home directory
- `$` → Normal user | `#` → Root user

What is a Shell Script?

- A text file containing multiple shell commands executed sequentially.

Why use it?

✅ Automate repetitive tasks
✅ Ensure consistency
✅ Improve efficiency

Example:

```
#!/bin/bash
date
pwd
ls
```

Understanding the Shebang Line (`#!/bin/bash`)

Tells the system which interpreter should execute the script.

Syntax:

```
#!/bin/bash
```

Explanation:

- `#!` → Shebang (Hash + Bang)
- `/bin/bash` → Path to the Bash interpreter

Why Important?

- Ensures portability
- Prevents execution errors

Hands-on

Comments in Shell Script

Purpose: Explain, document, or disable code parts.

Single-line Comment:

```bash
# This is a single-line comment
```

Multi-line Comment:

```bash
: '
This is a multi-line comment
'
```

Variables in Shell

Definition: Named containers to store data (strings, numbers, paths, etc.)

Rules:

1. No space around `=`
2. Cannot start with a number
3. Use `$` to access values

Example:

```
username="nahid"

echo $username
```

Accessing Variables

```
name="Alice"

echo "Hello $name"

echo "Hello ${name}smith"
```

✅ `${variable}` prevents confusion with adjacent text.

Command Example:

target_ip="192.168.1.1"

nmap $target_ip

Read-only Variables

Use Case: Prevent modification of sensitive or fixed values.

Example:

readonly API_KEY="abc123xyz789"

API_KEY="new_key" # Error

Output:

readonly variable

Variable Types

Type	Scope	Example
Local	Current shell/session only	`local_var="hello"`
Environment	Inherited by all processes	`export PATH`
Shell	Built-in to shell	`$USER`, `$HOME`, `$PATH`

Special Shell Variables

Variable	Description
`$0`	Script name
`$1`	First argument
`$$`	Current process ID
`$?`	Exit status of last command

Conditionals & Loops

Conditional Statements (`if` Statement)

Purpose: Allow scripts to make decisions based on conditions.

Concept:

> *If the condition is true → execute this block*
> *Otherwise → execute the alternate block*

Example Scenarios:

- Check if a **file exists**
- Verify if a **service is running**
- Validate **logged-in user**

- Confirm **port status**

Basic `if` Structure

```
if [ condition ]; then
    # Commands when condition is true
else
    # Commands when condition is false
fi
```

Key Notes:
✅ Always include spaces after `[` and before `]`
✅ Use `then` in the same or next line
✅ Close block with `fi`

Introduction to Loops

Purpose: Repeat a set of commands multiple times without manual repetition.

Why Use Loops?

- Process multiple **files or IPs**
- Monitor **multiple users**
- Automate **repeated scans or checks**

`for` Loop Syntax

```
for variable in list
do
    # Commands
done
```

Exit Status `$?`

`$?` = Exit status of the last executed command

Value	Meaning
`0`	Command executed successfully
Non-zero	Error or failure

Script Arguments & Environment Variables

What Are Script Arguments?

- Arguments are **values passed to a script during execution**
- Make scripts **dynamic and reusable**
- Reuse same script with different inputs

Example:

```
./scan.sh 192.168.1.1
```

Here, `192.168.1.1` is an **argument**

Special Argument Variables

Variable	Description
`$0`	Script name
`$1`, `$2`, `$3`	Positional arguments
`$#`	Total number of arguments
`$@`	All arguments (as separate words)
`$*`	All arguments (as single word)

Argument Demo

Script:

```
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "Number of arguments: $#"
echo "All arguments: $@"
```

Run:

```
./arguments.sh 192.168.1.1 80
```

Environment Variables

- Predefined system variables available to all processes
- Store system configuration and user session details
- Example: `$USER`, `$HOME`, `$PATH`

Why Use Environment Variables?

✅ Access system info dynamically
✅ Store configuration securely
✅ Build environment-specific scripts (Dev/Prod)

Example:

```
echo "User: $USER"
echo "Home: $HOME"
echo "Terminal: $TERM"
```

Common Environment Variables

Variable	Description
`$USER`	Current username
`$HOME`	User's home directory
`$PWD`	Present working directory
`$PATH`	System executable path
`$SHELL`	Current shell
`$TERM`	Terminal type

Functions, Return Codes & Exit Status

Functions?

- A **function** is a **named block of code** that performs a specific task
- Helps avoid **repetition** and keeps the script **clean and modular**
- Defined once, reused multiple times

Example:

```
my_function() {
  echo "This is a function"
}
my_function
```

Why Use Functions?

✅ Improves script readability
✅ Reduces code duplication
✅ Easy debugging and maintenance

Return Codes & Exit Status

Concept	Description
Return Code	The numeric value returned from a function
Exit Status	The result of the last executed command
Convention	`0` = Success, non-zero = Failure

File Handling, Redirection & Pipelines

What Is File Handling?

- File handling means **creating, reading, appending, and deleting files**
- It allows automation scripts to **store and manage data efficiently**

Example:
You can create log files or store scan results for later analysis.

Why File Handling Matters

✅ Store command results for future review
✅ Automate file updates without manual editing
✅ Maintain security or scan logs easily
✅ Enable real-time monitoring through log files

Key Concepts:

Operator	Meaning
`>`	Create new file / overwrite existing
`>>`	Append to existing file
`cat`	Display file content

What Is Redirection?

- Redirection means sending command output **to a file instead of the screen**
- Helps in **saving command results or logs**

📘 Example:

```
date > date.txt        # Save output to file
ls > files.txt         # Save list of files
ls /fakepath 2> error.txt  # Save errors
```

Types of Redirection:

Symbol	Description
`>`	Redirect STDOUT (output)
`>>`	Append STDOUT
`2>`	Redirect STDERR (error)
`&>`	Redirect both STDOUT & STDERR

What Are Pipelines?

- A **pipeline (|)** connects multiple commands
- The **output of one command** becomes the **input of another**

Example of Using Pipelines

```
ls | wc -l              # Count number of files
cal | head -1           # Show only first line of calendar
ls | grep txt           # Filter txt files
df -h | grep /dev/sda   # Show specific disk info
```

Scheduling Tasks (Cron Jobs)

🕒 **Cron** is a background service in Linux used to **run tasks automatically** at scheduled times.

Why Use Cron Jobs?

✅ Automate repetitive tasks
✅ Schedule system maintenance or backups
✅ Run monitoring or alerting scripts automatically
✅ Save time & ensure consistency in execution

Cron Job Syntax

Crontab Command:

```
crontab -e
```

Syntax Format:

```
* * * * * command_to_execute
| | | | |
| | | | └ Day of week (0-6)
| | | └── Month (1-12)
| | └──── Day of month (1-31)
| └────── Hour (0-23)
└──────── Minute (0-59)
```

Cron job calculations site: https://crontab.guru/

Logging, Error Handling & Best Practices

Why Logging Is Important

📘 **Definition:**
Logging tracks what your script did — success, failure, or unexpected errors.

💡 **Benefits:**

- Debug issues faster
- Audit what ran and when
- Identify failures automatically

Best Practices

✅ Use `set -e` → Stop script if any command fails
✅ Use `set -u` → Error out on undefined variables
✅ Redirect output and errors:

- `>` → Standard output
- `2>` → Error
- `>>` → Append logs

✅ Always log timestamps

Simple Logging Script

```
#!/bin/bash
echo "Script started" >> script.log
echo "Doing task 1..." >> script.log
mkdir test_folder >> script.log 2>&1
echo "Doing task 2..." >> script.log
cp *.txt test_folder/ >> script.log 2>&1
echo "Script finished" >> script.log
```

Run:

```
./simple_log.sh
cat script.log
```

Error Handling

Code Example:

```
#!/bin/bash
echo "Starting script..." > output.txt

ls /nonexistent_folder > output.txt 2> error.txt

if [ $? -eq 0 ]; then
   echo " Last command worked" >> output.txt
else
   echo " Last command failed" >> output.txt
fi

cat output.txt
cat error.txt
```

- `$?` checks last command's exit status
- `0` = success, Non-zero = failure
- Errors redirected to `error.txt`

Use Cases

✅ Automate **system backups**
✅ Monitor **security log rotation**
✅ Run **vulnerability scans** at night
✅ Generate **daily system reports**
✅ Schedule **cleanup tasks** (old logs/temp files)

Text / Data Parsing with `grep`, `awk`, `sed`, `jq`

`grep` (Global Regular Expression Print)

Fast pattern search tool for use like filtering logs, detecting errors, or searching text streams.

Common examples

```
grep "ERROR" /var/log/syslog            # exact match
grep -i "error" /var/log/syslog          # case-insensitive
grep -n "Failed" /var/log/auth.log       # show line numbers
grep -R "timeout" /var/log/              # recursive search
```

awk (Field / Column Processing)

- Mini scripting language for field-based processing.
- Extract columns, aggregate, and reformat tabular output.

Basic usage

```
awk -F',' '{print $2, $4}' users.csv        # print column 2 and 4 (CSV)
ps aux | awk '{print $11, $3, $4}' | head    # command, CPU, MEM
```

Power examples

```
awk -F',' '$4=="devops" {print $2 "," $3}' users.csv   # conditional extract
awk '{sum += $3} END {print sum}' file.txt            # simple aggregation
```

Use `-F` to set the delimiter, use `$1, $2...` for fields.

---

sed (Stream Editor)

Non-interactive editor for replace/delete/insert operations

Common patterns

```
sed 's/old/new/g' input.txt          # show replacement
sed -i 's/old/new/g' config.yml       # edit file in-place
sed -n '1,10p' file.txt              # print lines 1-10
```

sed -i '/^#/d' file.txt # delete comment lines
sed -i '/WARN/d' app.log # delete lines containing WARN

sed for config update

Replace `ENV=dev` with `ENV=prod` in `app.env` and backup.

cp app.env app.env.bak
sed -i 's/^ENV=.*/ENV=prod/' app.env

`^ENV=.*` anchors to line start, `.*` matches rest — safe replacement.

`jq` (JSON Processor)

Command-line JSON parser & transformer for Parse API responses (curl), extract JSON fields, reformat JSON.

Examples

```
jq '.' sample.json                          # pretty-print
jq '.city' sample.json                      # extract field
curl -s api.example.com/data | jq '.items[] | {id,name}'
jq -r '.items[].ip' response.json           # raw output list
```

jq '.' sample.json # pretty-print
jq '.city' sample.json # extract field
curl -s api.example.com/data | jq '.items[] | {id,name}'
jq -r '.items[].ip' response.json # raw output list

Logs: grep + awk

Show top 10 memory-using processes with names.

ps aux | sort -nrk 4 | awk '{print $11, $4}' | head -10

Explain:

- `ps aux` → list processes
- `sort -nrk 4` → sort by column 4 (MEM %) descending
- `awk` → print command + memory column

Combining Tools

Problem: From `app.log`, find ERROR entries, replace ERROR→CRITICAL, extract timestamps.

```
grep "ERROR" app.log \
  | sed 's/ERROR/CRITICAL/g' \
  | awk '{print $1, $2}' > critical_times.txt
```

Explain:

- `grep` filters lines → `sed` modifies text → `awk` extracts date/time.